Machine Learning for Traffic Flow Prediction in Urban Areas

Apeksha Ashok (PES1UG24AM048), Arnav Choudhary (PES1UG24AM050)

1. Problem Statement

Urban congestion costs time, fuel and safety. Accurate short-term traffic volume forecasts let signal control, routing and incident response act before a jam forms. We predict the hourly vehicle count on an urban interstate segment from calendar, weather and recent-traffic information. The project has the same purpose as the Stanford CS229 (Spring 2021) report *Machine Learning for Traffic Flow Prediction in Urban Areas* (Huang and Zhou), which predicted a five-level congestion label for 35 Manhattan roads from road, day and hour using a one-hidden-layer neural network (76.8% validation accuracy). We differ in both data and method: we use the public UCI Metro Interstate dataset, and we compare tree-based and linear models against forecasting baselines and a neural net. We solve two tasks: (1) **congestion-level classification** (the paper's task) and (2) **traffic volume regression** (a finer-grained view).

2. Dataset

We use the UCI *Metro Interstate Traffic Volume* dataset: hourly westbound I-94 traffic (Minneapolis-St Paul) joined with weather and US holiday data. After removing duplicate timestamps, rebuilding a regular hourly grid and dropping rows without a target or lag history, we have **33,367** hourly samples from 2012-10-09 to 2018-09-30. Target: *traffic_volume* (mean 3266, max 7280 vehicles/hour). Raw features: temperature, rain, snow, cloud cover, weather category, holiday flag and timestamp. Known sensor errors (0 K temperatures and implausible rain spikes) were treated as missing. The data is split chronologically: first 26,693 samples for training, last 6,674 for testing, so models never see the future.

3. Approach

Features. (a) Calendar: hour (raw and sin/cos), day of week, month, weekend and holiday flags. (b) Weather: temperature, rain, snow, cloud cover, one-hot weather category. (c) Lags: traffic 1, 2, 3, 24 and 168 hours earlier (recent trend, same hour yesterday and last week). This gives 22 features without lags and 27 with.
Models. Two baselines: historical mean per (day-of-week, hour) and persistence (previous hour). Learned models: Ridge regression, Random Forest and Gradient Boosting, each trained on both feature sets to separate what calendar/weather explains from what recent traffic adds.
Congestion levels. As in the paper, congestion is a label from 0 (free flow) to 4 (severe). We define it as hourly volume relative to the maximum training volume, in 20% bands (level edges 1456, 2912, 4368, 5824 vehicles/hour). Classifiers: Logistic Regression, Random Forest, Gradient Boosting and an MLP, plus a paper-style MLP (4 hidden neurons, day and hour only) as a reference.
Metrics. Regression: MAE, RMSE (vehicles/hour), R^2. Classification: accuracy, within-one-level accuracy (off by at most one level) and macro-F1. All on the held-out final period.

4. Implementation

Python 3 with pandas, scikit-learn and matplotlib. `src/data.py` downloads and cleans the data and builds features; `src/train.py` runs the chronological split, fits all models with fixed seeds, and writes metrics and the plot; `src/make_writeup.py` builds this report from the saved results. Random Forest uses 200 trees (min leaf 3); Gradient Boosting uses histogram-based trees (300 iterations, learning rate 0.08); Ridge uses standardized inputs, alpha 1. The classifiers use matching settings (Random Forest 200 trees, Gradient Boosting 200 iterations, Logistic Regression up to 2000 iterations). The MLP has one hidden layer of 32 units (4 units for the paper-style reference), standardized inputs, early stopping and at most 500 iterations. The whole pipeline reproduces with two commands (see the README).

5. Results

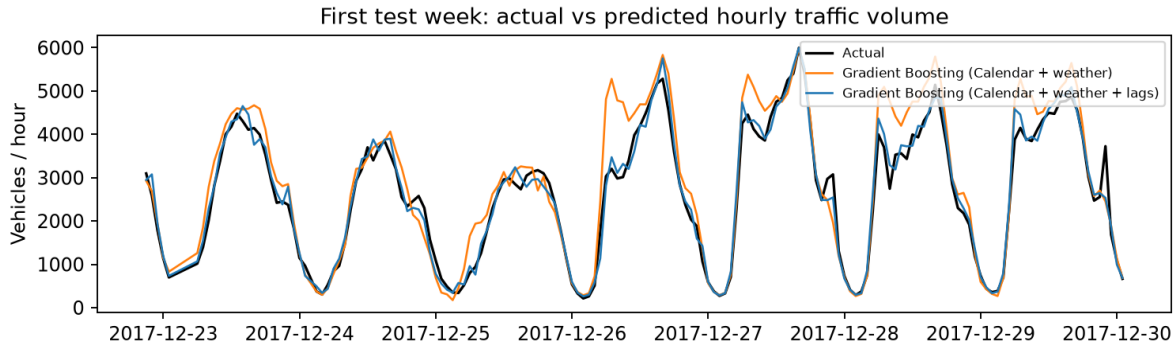
Congestion-level classification (5 classes, the paper's task):

Feature set	Model	Accuracy	Within-1	Macro-F1
Baseline	Majority level	29.4%	59.6%	0.091
Baseline	Historical mean -> level	80.7%	99.0%	0.784
Baseline	Persistence -> level	66.4%	96.4%	0.623
Calendar + weather	Logistic Regression	67.6%	96.9%	0.559
Calendar + weather	Random Forest	83.1%	99.1%	0.815
Calendar + weather	Gradient Boosting	83.9%	99.3%	0.825
Calendar + weather	Neural Net (MLP, 32 hidden)	81.3%	99.3%	0.794
Calendar + weather + lags	Logistic Regression	85.2%	99.8%	0.836

Feature set	Model	Accuracy	Within-1	Macro-F1
Calendar + weather + lags	Random Forest	90.0%	100.0%	0.889
Calendar + weather + lags	Gradient Boosting	89.5%	100.0%	0.883
Calendar + weather + lags	Neural Net (MLP, 32 hidden)	88.0%	99.9%	0.867
Day + hour only	Paper-style MLP (4 hidden)	66.4%	93.9%	0.551

Traffic volume regression (vehicles/hour):

Feature set	Model	MAE	RMSE	R^2
Baseline	Historical mean (day-of-week x hour)	280	497	0.935
Baseline	Persistence (previous hour)	584	807	0.830
Calendar + weather	Ridge	808	1020	0.729
Calendar + weather	Random Forest	236	393	0.960
Calendar + weather	Gradient Boosting	228	377	0.963
Calendar + weather + lags	Ridge	285	404	0.957
Calendar + weather + lags	Random Forest	151	239	0.985
Calendar + weather + lags	Gradient Boosting	146	225	0.987



6. Conclusions

The best model was **Gradient Boosting with lag features** (RMSE 225, R^2 0.987), which cuts RMSE by 72% against the persistence baseline (807). Using only calendar and weather information, the best model (Gradient Boosting) reaches RMSE 377; adding lag features lowers it by a further 40%, showing that recent traffic carries information that time of day and weather cannot. The historical-mean baseline (RMSE 497) captures the daily and weekly rhythm but misses day-specific deviations. For five-level congestion classification, the best model (Random Forest with lags) reaches 90.0% accuracy (100.0% within one level), versus 80.7% for the historical-mean baseline and 66.4% for a paper-style MLP on day and hour only; the paper reports 76.8% on a different dataset, so the numbers are indicative rather than directly comparable. Limitations: a single sensor, one corridor, and no incident or event data. Future work: multi-step horizons, spatial neighbours from several sensors, and sequence models (LSTM / graph networks).